MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Daniel Moisei FAF-241

Report

Laboratory Work No. 6

Operating Systems

Topic: Inside the xv6 File System

Checked by:
Reiţman Patricia, *university assistant*

Chişinău - 2026

CONTENTS

Introduction **3**

1 Reading What xv6 Shows **4**

1.1 Types, inode numbers, and directory entries 4

2 The mystat Program and Hard Links **5**

2.1 Implementation 5

2.2 Observed hard-link behaviour 6

3 Measured File System Limits **7**

3.1 Name and file-size limits 7

3.2 Disk layout 7

4 Recursive find **8**

4.1 Implementation 8

4.2 Fresh-disk test 9

Conclusion **11**

INTRODUCTION

The purpose of this laboratory work was to inspect the small xv6 file system from the user level and connect its visible behaviour with the kernel data structures that implement it. I decoded the fields printed by `ls`, implemented `mystat` to inspect inodes and hard links, measured the layout and file-size limits declared in `kernel/fs.h`, and implemented a recursive `find` program.

All demonstrations were run in the configured `xv6-riscv` tree through QEMU. The report contains terminal captures created from those completed sessions. The tree used 1024-byte blocks, 14-byte directory-name fields, 12 direct block pointers, and a 256-entry indirect block; therefore the observed and source-level results agree.

1 READING WHAT XV6 SHOWS

1.1 Types, inode numbers, and directory entries

The format printed by xv6 `ls` is *name, type, inode number, size*. In `kernel/stat.h`, the constants are `T_DIR = 1`, `T_FILE = 2`, and `T_DEVICE = 3`. Thus the entries `.` and `..` are directories, `README` is a regular file, and `console` is a device. Figure 1.1 also records the output of the implemented `mystat` command for one object of each type.

The root directory has inode number 1, as specified by `ROOTINO` in `kernel/fs.h`. At the root, both `.` and `..` have inode 1 because the parent of the root is defined to be the root itself. The newly created empty directory `a` has a 32-byte size. A `struct dirent` contains a 2-byte inode number and a 14-byte name, so it occupies 16 bytes. Its two initial entries are `.` and `..`; therefore $2 \times 16 = 32$ bytes.

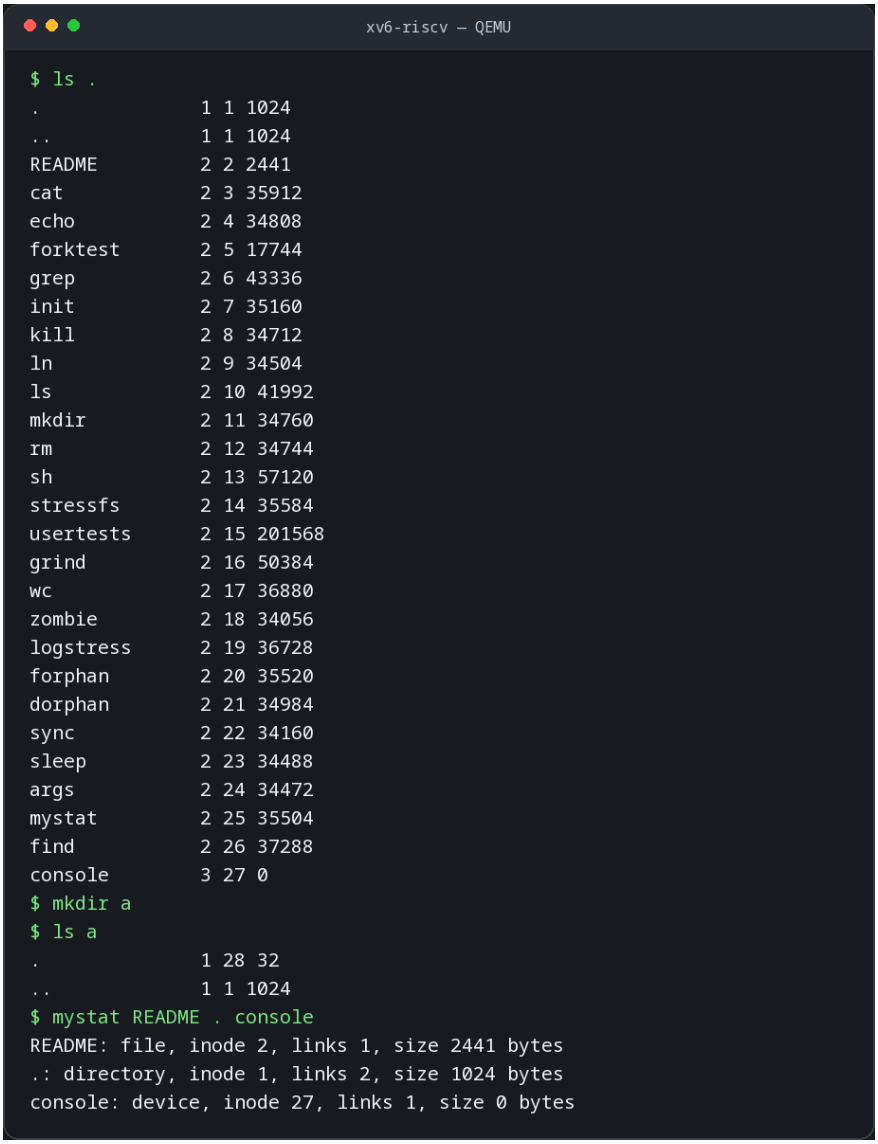


Figure 1.1: xv6 directory listing, empty-directory size, and type decoding with mystat

2 THE MYSTAT PROGRAM AND HARD LINKS

2.1 Implementation

The new program calls `stat()` for each path. It maps the numeric `type` field to a readable label and prints the inode, link count, and byte size. It continues after an invalid path so one error does not prevent the remaining requested paths from being inspected.

Listing 1: Implementation of user/mystat.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int type)
{
  switch(type) {
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
  case T_DEVICE:
    return "device";
  default:
    return "?";
  }
}

int
main(int argc, char *argv[])
{
  struct stat st;
  int i;

  if(argc < 2) {
    fprintf(2, "usage: mystat <path> ...\n");
    exit(1);
  }

  for(i = 1; i < argc; i++) {
    if(stat(argv[i], &st) < 0) {
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
```

```
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

The xv6 `Makefile` was extended with the two tab-indented `UPROGS` entries below, which caused the programs to be included in the file-system image.

```
    $U/_mystat\
    $U/_find\
```

2.2 Observed hard-link behaviour

After `ln README r2`, both names referred to inode 2, retained the size of 2441 bytes, and showed link count 2. After `rm README`, `r2` still referred to inode 2 and its data remained readable; only the link count changed from 2 to 1. The following `cat r2 | wc` result of 2441 bytes confirms that unlinking the first name did not remove the file data.

In `kernel/sysfile.c`, `sys_unlink()` clears the directory entry and then executes `ip->nlink--` followed by `iupdate(ip)`. The data blocks are reclaimed only when the inode's link count is zero and its in-memory reference count reaches zero; that final release is handled by the inode cleanup path. xv6 therefore preserves the data while another directory entry or an open file still refers to the inode.

The on-disk `struct dinode` contains `type`, `major`, `minor`, `nlink`, `size`, and `addrs[NDIRECT+` Unlike a modern Linux inode, it contains no owner, group, mode, permission, or symbolic-link metadata. xv6 has no symbolic links and no permission checks.

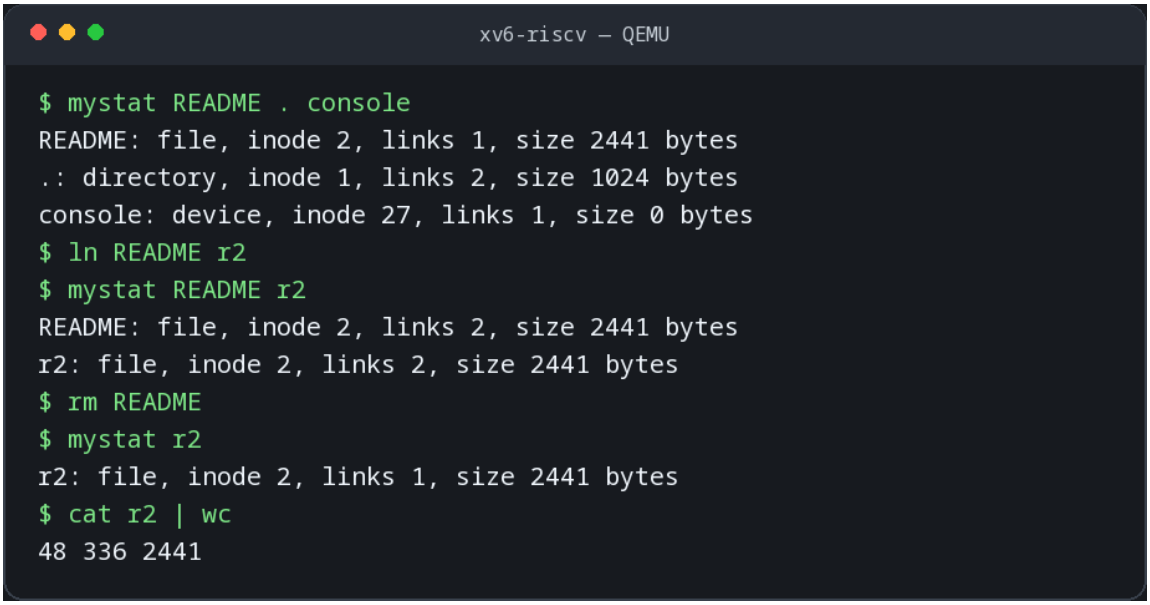


Figure 2.1: Hard-link creation, removal of one name, and preserved file contents

3 MEASURED FILE SYSTEM LIMITS

3.1 Name and file-size limits

The constants read from `kernel/fs.h` were `BSIZE = 1024`, `NDIRECT = 12`, `NINDIRECT = BSIZE / sizeof(uint)`, and `DIRSIZ = 14`. Creating `averyveryverylongname` and listing the directory displayed `averyveryveryl`. The name has been truncated to the first 14 characters, exactly as `DIRSIZ` requires.

Each indirect-block entry is a 4-byte block number, so the indirect block holds $1024/4 = 256$ addresses. The largest file uses 12 direct blocks and 256 indirect blocks:

$$(12 + 256) \times 1024 = 274\,432 \text{ bytes} = 268 \text{ KiB}.$$

3.2 Disk layout

The build output reported `nmeta 47`: 1 boot block, 1 superblock, 31 log blocks, 13 inode blocks, and 1 bitmap block. The whole image contains 2000 blocks. Subtracting the 47 metadata blocks leaves 1953 data blocks, which is also reported by `mkfs`. The commands and their actual output are shown in Figure 3.1.

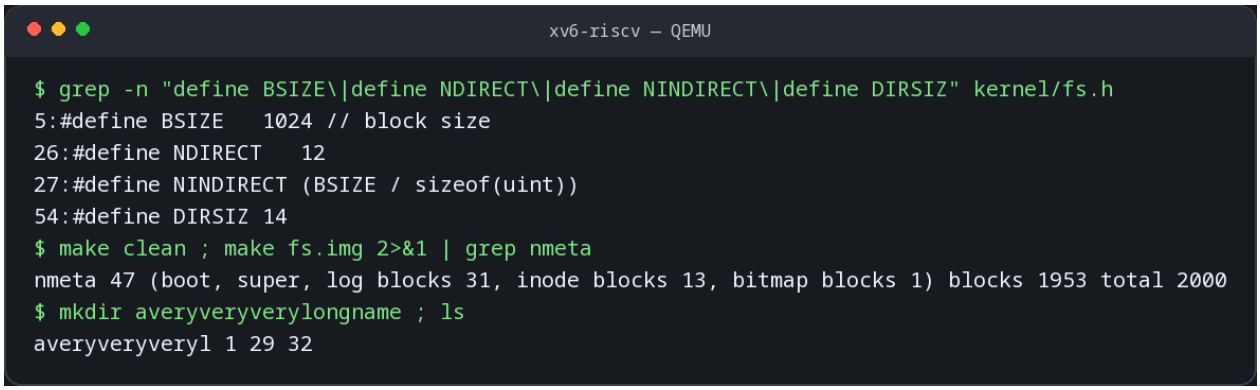


Figure 3.1: Constants, disk-layout summary, and the observed 14-character directory name

4 RECURSIVE FIND

4.1 Implementation

The `find` utility opens the supplied directory, reads fixed-size `dirent` records, skips free records and the special `./..` names, and constructs each full child path. It prints a path if its final component equals the requested target, then recursively visits child directories. Every opened file descriptor is closed before returning, which is necessary because xv6 processes have a small open-file limit.

Listing 2: Implementation of user/find.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"
#include "kernel/fcntl.h"

void
find(char *path, char *target)
{
  char buf[512], name[DIRSIZ + 1], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if(fstat(fd, &st) < 0) {
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  if(st.type != T_DIR) {
    close(fd);
    return;
  }
  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)) {
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }
```

```
  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)) {
    if(de.inum == 0)
      continue;
    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
      continue;
    memmove(p, name, DIRSIZ + 1);
    if(stat(buf, &st) < 0) {
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }
    if(strcmp(name, target) == 0)
      printf("%s\n", buf);
    if(st.type == T_DIR)
      find(buf, target);
  }
  close(fd);
}

int
main(int argc, char *argv[])
{
  if(argc != 3) {
    fprintf(2, "usage: find <dir> <name>\n");
    exit(1);
  }
  find(argv[1], argv[2]);
  exit(0);
}
```

4.2 Fresh-disk test

The test was performed after rebuilding the disk image. The tree contains three entries called `r2`: two new regular files and a hard link to `cat`. The command printed all three matching paths. The order is a directory traversal order and is not part of the specification. The second query correctly printed the directory `a/b`.

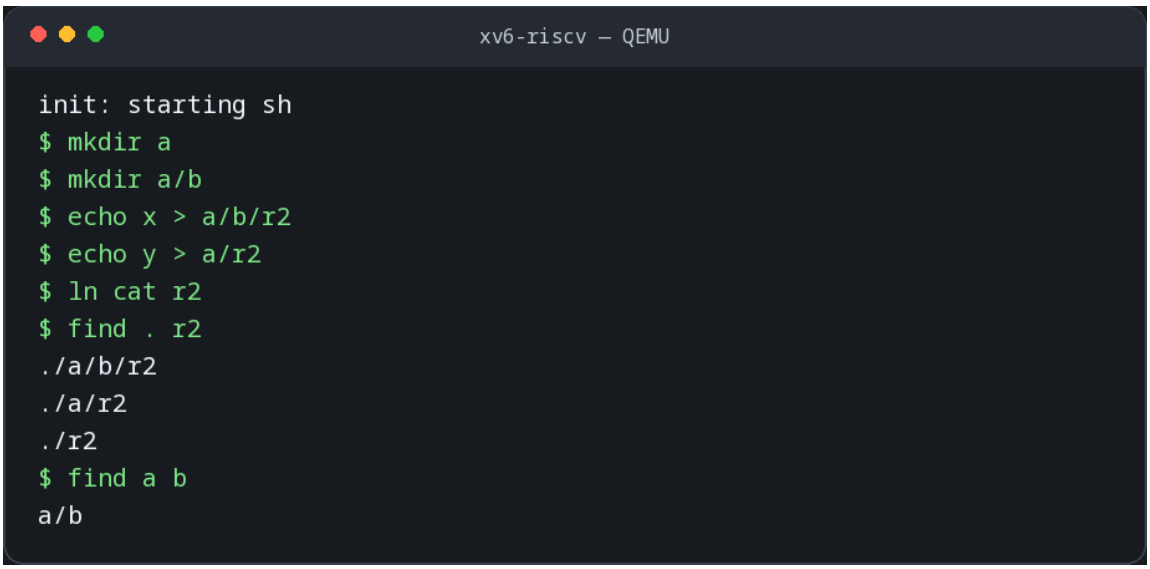


Figure 4.1: Fresh xv6 session testing recursive find

CONCLUSION

This laboratory work showed the direct correspondence between xv6 file-system structures and observable user commands. Directories are files made of 16-byte directory entries; names point to inode numbers; and hard links are multiple names for one inode, not duplicate data. The source constants explain both the 14-character name limit and the 268 KiB maximum file size. The completed `mystat` and `find` utilities compiled successfully, were included in the xv6 image, and were verified in QEMU using the required link and recursive-search scenarios.